



UNIVERSITÀ
DEGLI STUDI
FIRENZE

SmartVend: Sistema di gestione smart per vending machines

Anno 2025

Matteo Minin, Simone Pellicci, Simone Suma

Professore: Enrico Vicario

Corso di laurea triennale in Ingegneria Informatica

Indice

1	Introduzione	3
1.1	Statement	3
2	Progettazione	3
2.1	Use-Case Diagrams	3
2.2	Use-Case Templates	5
2.2.1	Casi d'Uso del Customer	5
2.2.2	Casi d'Uso del Worker	6
2.2.3	Casi d'Uso dell'Admin	7
3	Progettazione del Database	8
3.1	Schema Entità-Relazione (ER)	8
3.2	Analisi Dettagliata delle Tabelle	9
3.2.1	Gestione degli Utenti	9
3.2.2	Gestione delle Macchine e dell'Inventario	9
3.2.3	Gestione delle Transazioni	9
3.2.4	Gestione Operativa	10
3.3	Scelte Progettuali e Integrità dei Dati	10
4	Progettazione dell'Interfaccia Utente (Mockups)	10
4.1	Flusso di Accesso e Registrazione	11
4.2	Dashboard per Ruolo Utente	11
4.3	Catalogo Prodotti e Acquisto	13
5	Implementazione e Testing	14
5.1	Architettura e Strumenti di Sviluppo	14
5.2	Livello di Persistenza: DAO e ORM Hibernate	15
5.3	Livello di Logica di Business (Service)	16
5.4	Ambiente di Esecuzione con Docker	17
5.5	Strategia di Testing	18
5.5.1	Test di Unità e Mocking	18

5.5.2	Test di Integrazione	19
5.5.3	Analisi della Copertura del Codice	20

1 Introduzione

SmartVend è un software di gestione di una rete di vending machines smart che mira a rendere il processo di utilizzo, manutenzione e approvvigionamento dei prodotti più semplice ed efficiente possibile. Il software si occuperà di mantenere aggiornati i log riguardanti le vendite e i problemi riscontrati sui singoli distributori in modo da poter identificare problematiche comuni da risolvere su determinati modelli di distributori e tenere traccia dei prodotti che vengono più apprezzati in determinate zone geografiche.

1.1 Statement

L'obiettivo principale del sistema è fornire una soluzione per la gestione efficiente delle vending machines, con funzionalità specifiche per diverse tipologie di utenti: Clienti (Customer), Manutentori (Worker) e Amministratori (Admin).

L'applicazione consentirà all'utente di tipo **Customer** di acquistare prodotti connettendosi al distributore con il proprio dispositivo e di ricaricare il proprio borsellino elettronico pagando con carta o contanti.

L'utente di tipo **Worker** (Manutentore) avrà la possibilità di visualizzare l'elenco degli interventi a lui assegnati con la relativa descrizione del compito da svolgere e di notificare il completamento di una riparazione o approvvigionamento di un distributore. Potrà inoltre segnalare problemi o guasti.

L'utente di tipo **Admin** (Amministratore) avrà responsabilità più ampie, tra cui la gestione degli utenti (CRUD), inclusa l'aggiunta, la rimozione e l'aggiornamento dei dettagli dei manutentori. L'amministratore potrà anche gestire i distributori automatici (CRUD), consentendo l'aggiunta, la rimozione e l'aggiornamento delle loro informazioni, e gestire gli articoli disponibili nei distributori (CRUD), permettendo di aggiungere, rimuovere e aggiornare gli articoli. Infine, avrà accesso a statistiche sulle vendite e sulle performance delle macchine.

L'obiettivo primario è offrire un'esperienza utente intuitiva e funzionale, garantendo al contempo un'elevata manutenibilità e la possibilità di future espansioni. Il sistema è progettato per ottimizzare la gestione operativa dei distributori automatici, riducendo i tempi di inattività e migliorando l'efficienza complessiva del servizio.

2 Progettazione

2.1 Use-Case Diagrams

I tre attori principali che interagiscono con il sistema sono il Cliente (Customer), il Manutentore (Worker) e l'Amministratore (Admin). Tutti gli attori derivano dalla classe **Utente** che espone

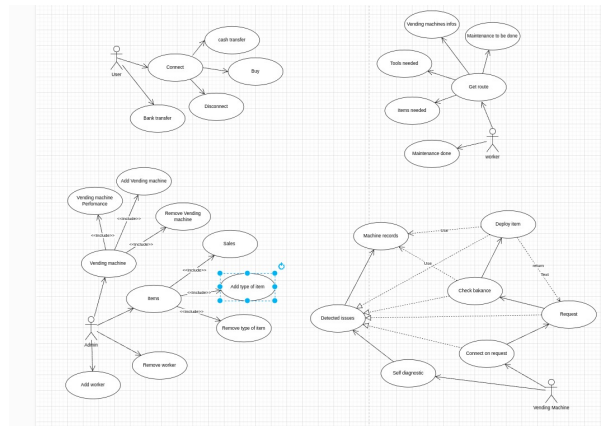


Figura 1: Use Cases

i metodi per fare l'accesso ("Login") e registrarsi ("Sign-Up") al sistema.

Il **Customer** una volta autenticato, può acquistare articoli ("Buy item"), connettersi al distributore automatico ("Connect to Vending Machine") e ricaricare il proprio account ("Recharge account").

Il **Worker** può visualizzare i compiti assegnati ("View Assigned tasks"), visualizzare i dettagli dei compiti ("View task details") e segnalare il completamento di un compito ("Report task completion"). Inoltre, tramite il sistema può segnalare un problema alla Vending Machine.

L'**Admin** ha il ruolo con più funzionalità. L'Admin può gestire la creazione, lettura, aggiornamento, eliminazione degli utenti ("Manage Users CRUD"), che include "Add worker" (aggiungere manutentori), "Remove User" (rimuovere utenti) e "Update User Details" (aggiornare i dettagli degli utenti), oltre a "View/Search User" (visualizzare/cercare utenti). Inoltre, l'Admin può gestire la creazione, lettura, aggiornamento, eliminazione dei distributori automatici ("Manage Vending Machines CRUD"), che comprende "Add Vending machine" (aggiungere distributori), "Remove Vending machine" (rimuovere distributori) e "Update Vending machine details" (aggiornare i dettagli dei distributori), e "View/Search Vending machine" (visualizzare/cercare distributori). L'Admin è anche responsabile della gestione della creazione, lettura, aggiornamento, eliminazione degli articoli ("Manage Items CRUD"), potendo "Add item" (aggiungere articoli), "Remove item" (rimuovere articoli) e "Update item" (aggiornare articoli), oltre a "View/Search item" (visualizzare/cercare articoli). Infine, l'Admin ha accesso a "View sales analytics" (visualizzare le analisi delle vendite) e "View machine performance analytics" (visualizzare le analisi delle performance delle macchine).

È presente anche un attore "Vending Machine" che può eseguire auto-diagnostica ("Run Self-diagnostic"), mostrare articoli ("Display item"), accettare richieste di connessione ("Accept connection request") e segnalare lo stato della macchina o eventuali problemi ("Report Status/issue").

2.2 Use-Case Templates

Si descrivono nel dettaglio alcuni dei casi d'uso più importanti, suddivisi per attore. È sottintesa la preconditione di Login prima di qualsiasi operazione sul gestionale.

2.2.1 Casi d'Uso del Customer

UC-1	Buy Item
Description	L'utente di tipo Customer acquista un prodotto da un distributore automatico a cui è connesso.
Level	User Goal
Main Actors	Customer
Basic Course	1. Il Customer si connette a un distributore automatico tramite l'applicazione mobile. 2. Il sistema mostra gli articoli disponibili e il loro prezzo. 3. Il Customer seleziona l'articolo desiderato. 4. Il sistema verifica la disponibilità dell'articolo nell'inventario del distributore e il saldo del Customer. 5. Se l'articolo è disponibile e il saldo è sufficiente, il sistema eroga l'articolo. 6. Il sistema aggiorna l'inventario del distributore e il saldo del Customer.
Alternative Paths	• 4a. Saldo insufficiente: Se il saldo del Customer non è sufficiente per l'acquisto, il sistema informa il Customer e propone di ricaricare l'account. Il Customer può scegliere di ricaricare (UC-2) o annullare l'acquisto. • 5a. Errore di erogazione: Se si verifica un errore durante l'erogazione dell'articolo, il sistema informa il Customer e registra un log di errore per la macchina. Il saldo del Customer non viene addebitato.

2.2.2 Casi d'Uso del Worker

UC-3	Report Task Completion
Description	L'utente di tipo Worker segnala il completamento di un'attività di manutenzione o rifornimento.
Level	User Goal
Main Actors	Worker
Basic Course	1. Il Worker visualizza l'elenco dei compiti assegnati. 2. Il Worker seleziona il compito che ha completato. 3. Il Worker conferma il completamento e, se necessario, aggiunge note. 4. Il sistema aggiorna lo stato del compito a "completato" nel log di manutenzione.
Alternative Paths	• 3a. Difficoltà nel completamento: Se il Worker riscontra difficoltà nel completare il compito, può indicare un aggiornamento dello stato a "in progress" o "pending" e aggiungere note per richiedere supporto o indicare la necessità di ulteriori interventi.

UC-4	Report Status/Issue (from Worker to Vending Machine)
Description	L'utente di tipo Worker segnala un problema o un guasto riscontrato su un distributore automatico.
Level	User Goal
Main Actors	Worker, Vending Machine
Basic Course	1. Il Worker identifica un problema o un guasto su un distributore. 2. Il Worker accede alla funzionalità di segnalazione problemi. 3. Il Worker descrive il problema (es. codice errore, messaggio) e, se possibile, indica il tipo di guasto. 4. Il sistema registra il problema nel log di rilevamento manutenzione [cite: 2, 6] e, se necessario, genera un nuovo compito di manutenzione.
Alternative Paths	• 4a. Problema duplicato: Se il problema è già stato segnalato e un compito è già in corso, il sistema notifica il Worker e gli chiede se vuole comunque aggiungere una nota al problema esistente.

2.2.3 Casi d'Uso dell'Admin

UC-5	Manage Users CRUD
Description	L'utente di tipo Admin gestisce gli account degli utenti (Clienti, Worker, Admin).
Level	User Goal
Main Actors	Admin
Basic Course	1. L'Admin accede alla sezione di gestione utenti. 2. L'Admin può visualizzare l'elenco degli utenti e cercare un utente specifico. 3. L'Admin può aggiungere un nuovo utente (es. un nuovo Worker). 4. L'Admin può modificare i dettagli di un utente esistente (es. ruolo, informazioni personali). 5. L'Admin può rimuovere un utente dal sistema.
Alternative Paths	• 3a. Utente esistente: Se si tenta di aggiungere un utente con un username o email già esistente, il sistema notifica l'Admin. • 5a. Utente con dipendenze: Se l'Admin tenta di rimuovere un utente con transazioni o compiti assegnati attivi, il sistema chiede conferma o blocca l'operazione fino alla risoluzione delle dipendenze.

UC-6	View Sales Analytics
Description	L'utente di tipo Admin visualizza statistiche e analisi sulle vendite dei prodotti.
Level	User Goal
Main Actors	Admin
Basic Course	1. L'Admin accede alla sezione di visualizzazione delle statistiche. 2. Il sistema presenta dati aggregati sulle vendite, come i prodotti più venduti, le fasce orarie di maggiore affluenza, e le entrate totali. 3. L'Admin può applicare filtri per periodo, distributore o prodotto.
Alternative Paths	• 3a. Nessun dato disponibile: Se per i filtri selezionati non ci sono dati di vendita, il sistema visualizza un messaggio appropriato.

3 Progettazione del Database

La persistenza dei dati è un requisito fondamentale per l'applicazione **JavaBrew**, in quanto garantisce che tutte le informazioni relative a utenti, macchine, prodotti e transazioni siano conservate in modo sicuro e coerente. Per soddisfare questa esigenza, è stato progettato un database relazionale, il cui schema è stato definito attraverso un modello Entità-Relazione (ER). Questa sezione illustra la struttura logica del database, descrivendo le entità principali, i loro attributi e le relazioni che le legano.

3.1 Schema Entità-Relazione (ER)

Il diagramma ER, mostrato nella Figura 2, fornisce una visione d'insieme completa della struttura del database. Questo modello è stato la guida per la creazione delle tabelle fisiche e per l'implementazione del livello di persistenza (DAO) nell'applicazione. Il design segue i principi di normalizzazione per ridurre la ridondanza dei dati e garantire l'integrità referenziale attraverso l'uso di chiavi primarie e chiavi esterne.

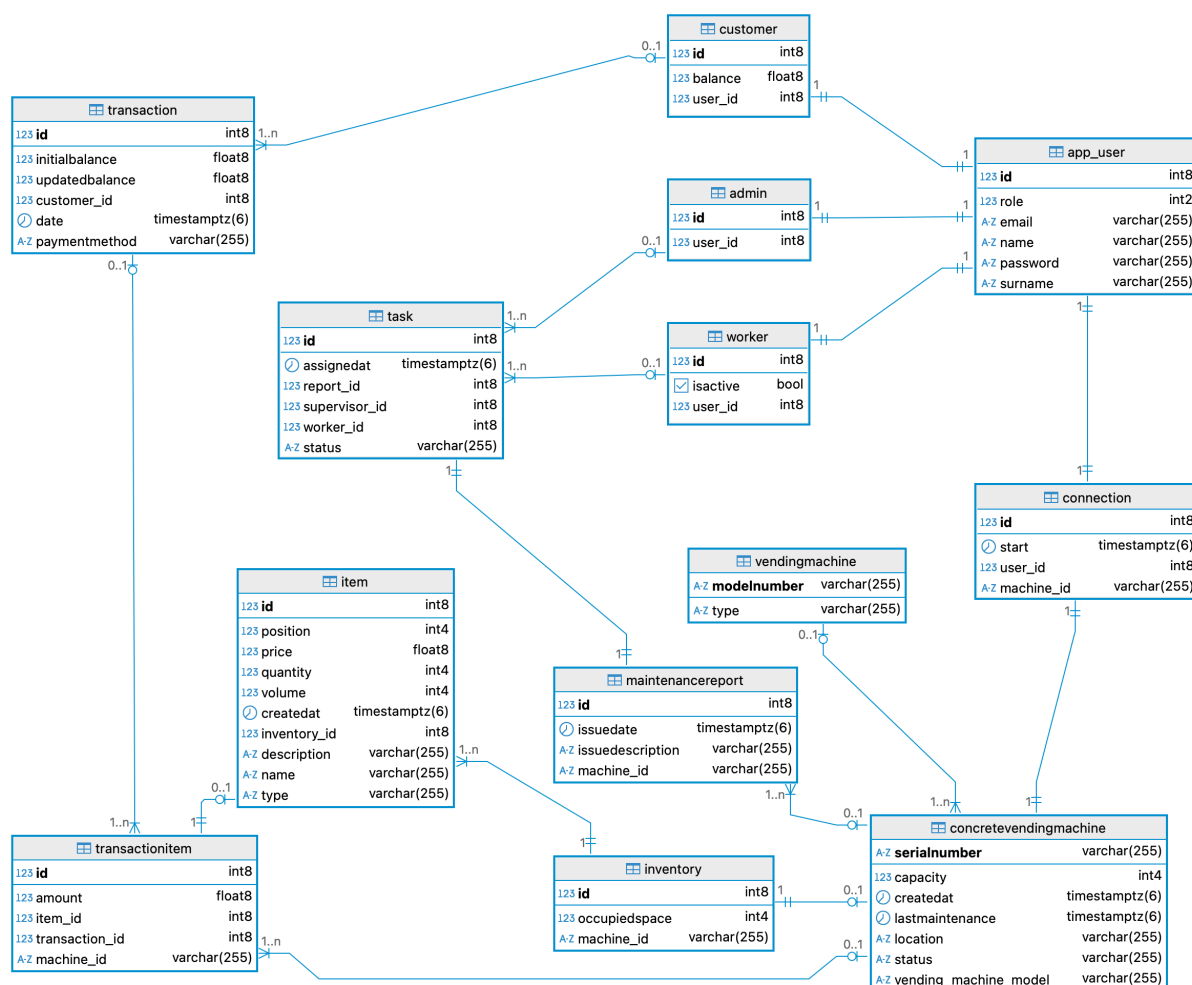


Figura 2: Diagramma Entità-Relazione (ER) del database di JavaBrew.

3.2 Analisi Dettagliata delle Tabelle

Lo schema è organizzato attorno a quattro aree funzionali principali: la gestione degli utenti, la gestione delle macchine, la registrazione delle transazioni e la gestione delle operazioni di manutenzione.

3.2.1 Gestione degli Utenti

La modellazione degli utenti adotta un approccio di **specializzazione**. Esiste una tabella base, `app_user`, che contiene gli attributi comuni a tutti i tipi di utente.

- **app_user**: Funge da genitore e memorizza le informazioni anagrafiche e di accesso fondamentali, come `id`, `email`, `password`, `name`, `surname` e `role`. Il campo `role` è cruciale per distinguere le diverse tipologie di utente.
- **customer**, **admin**, **worker**: Sono tabelle di specializzazione. Ognuna ha una relazione uno-a-uno con `app_user`, identificata dalla chiave esterna `user_id`. Questo design permette di estendere gli attributi specifici per ogni ruolo. Ad esempio, la tabella `customer` contiene il campo `balance` per il credito, mentre la tabella `worker` ha un flag `isactive`.

3.2.2 Gestione delle Macchine e dell'Inventario

Anche la modellazione delle macchine da caffè segue un principio di specializzazione, separando il concetto generico di macchina da quello concreto e installato.

- **vendingmachine**: Tabella che definisce i modelli generici di macchina, con attributi come `modelnumber` e `type`.
- **concretevendingmachine**: Rappresenta un'istanza fisica di una macchina installata in un determinato luogo. È collegata a un modello generico tramite la chiave esterna `vending_machine_model` e possiede attributi operativi come `serialnumber`, `location`, `status` e `capacity`.
- **inventory**: Ogni macchina concreta ha un inventario (relazione uno-a-uno). La tabella `inventory` tiene traccia dello spazio occupato (`occupiedspace`) e funge da collegamento per gli articoli.
- **item**: Contiene l'elenco dei prodotti disponibili, con attributi come `name`, `price`, `quantity` e `position`. È collegata all'inventario di una specifica macchina tramite la chiave esterna `inventory_id`.

3.2.3 Gestione delle Transazioni

Questa area è responsabile della tracciabilità di tutte le operazioni finanziarie.

- **transaction:** Memorizza l'instestazione di una transazione, come l'acquisto o una ricarica. Include l'ID del cliente (`customer_id`), il metodo di pagamento (`paymentmethod`), e il bilancio iniziale e finale (`initialbalance`, `updatedbalance`) per una facile verifica.
- **transactionitem:** Tabella di dettaglio che realizza la relazione multi-a-molti tra transazioni e articoli. Ogni record rappresenta una riga di uno scontrino, collegando un `transaction_id` a un `item_id` e specificando l'importo (`amount`).

3.2.4 Gestione Operativa

Questo gruppo di tabelle supporta le operazioni quotidiane dell'applicazione.

- **connection:** Traccia le connessioni attive tra un cliente e una macchina, memorizzando `user_id`, `machine_id` e l'orario di inizio (`start`).
- **task e maintenancereport:** Supportano il flusso di lavoro dei tecnici. Un `task` (compito) viene assegnato a un `worker` e può generare un `maintenancereport` (rapporto di manutenzione) una volta completato.

3.3 Scelte Progettuali e Integrità dei Dati

La progettazione del database è stata guidata da due principi chiave: la **normalizzazione** e l'**integrità referenziale**.

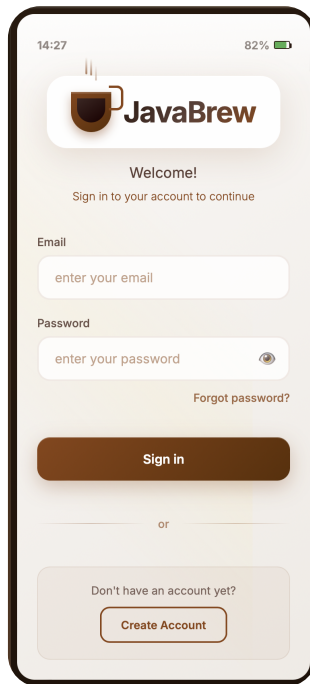
- **Normalizzazione:** Lo schema è stato progettato per raggiungere almeno la Terza Forma Normale (3NF), riducendo al minimo la ridondanza dei dati. Ad esempio, le informazioni di un utente sono memorizzate una sola volta nella tabella `app_user`, evitando duplicazioni.
- **Integrità Referenziale:** L'uso estensivo di chiavi esterne (`foreign key`) garantisce che le relazioni tra le tabelle siano sempre valide. Ad esempio, non è possibile creare una transazione per un cliente che non esiste, o assegnare un compito a un tecnico non presente nel sistema. Questi vincoli, imposti a livello di database, proteggono i dati da inconsistenze e sono fondamentali per la robustezza dell'intera applicazione.

4 Progettazione dell'Interfaccia Utente (Mockups)

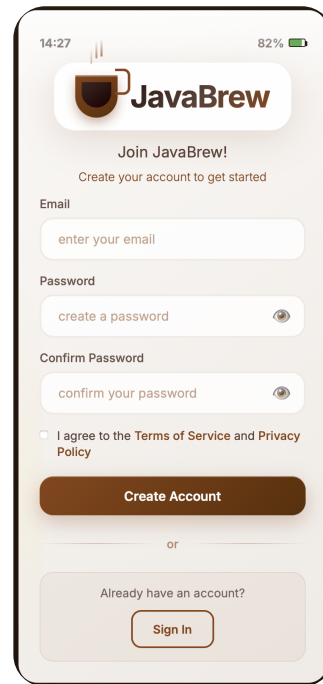
In questa sezione viene illustrata la progettazione dell'interfaccia utente (UI) per l'applicazione **JavaBrew**. I mockup presentati di seguito non si limitano a definire l'estetica del sistema, ma rappresentano l'architettura dell'interazione uomo-macchina (HCI), dettagliando i flussi operativi previsti per ciascun attore. L'obiettivo è fornire una chiara visualizzazione dell'esperienza utente (UX), dimostrando come il design supporti le funzionalità del sistema in modo intuitivo ed efficiente.

4.1 Flusso di Accesso e Registrazione

Il flusso di onboarding inizia con una schermata di login minimale (Figura 3a) e prosegue con un modulo di registrazione (Figura 3b) progettato per essere rapido e trasparente. Le interfacce sono pulite, richiedono solo le informazioni essenziali e guidano l'utente in modo chiaro.



(a) Schermata di login.



(b) Schermata di registrazione.

Figura 3: Interfacce per il flusso di accesso e registrazione.

4.2 Dashboard per Ruolo Utente

Ogni attore del sistema dispone di una dashboard personalizzata. La dashboard dell'Amministratore (Figura 4) è un centro di controllo strategico con una visione d'insieme. Le interfacce per il Cliente e il Tecnico (Figura 5) sono ottimizzate per l'uso mobile.

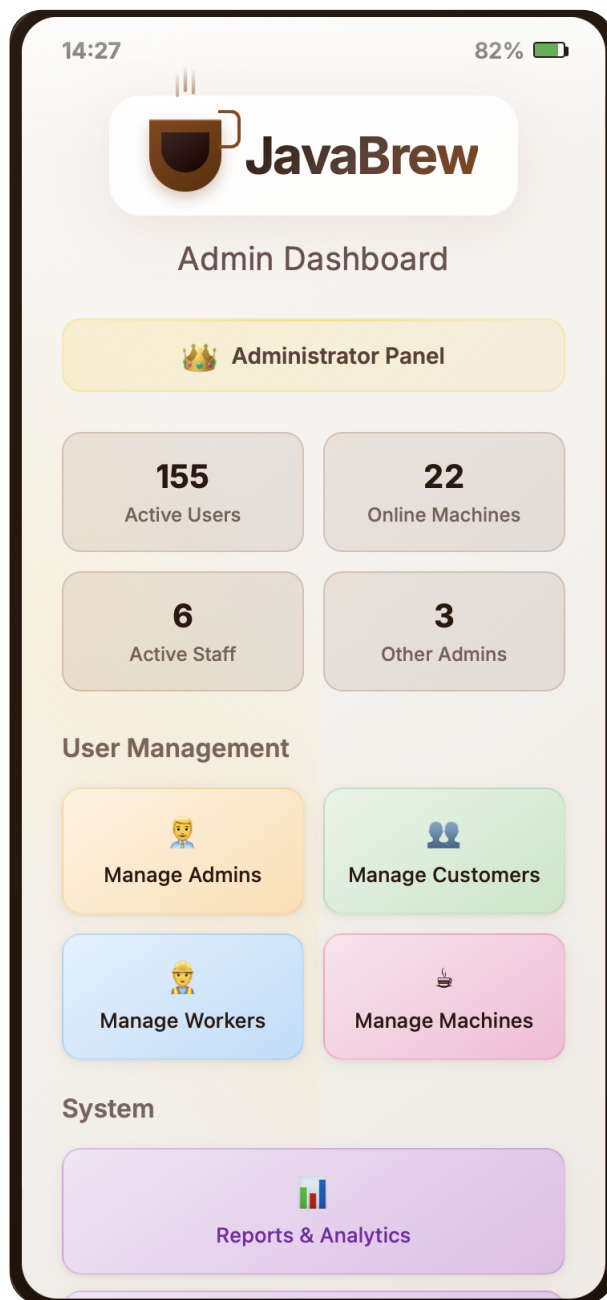
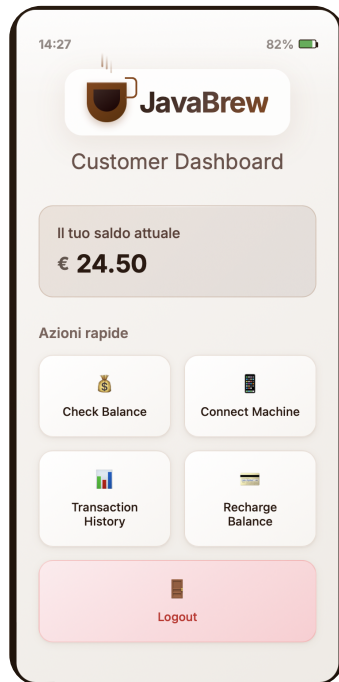
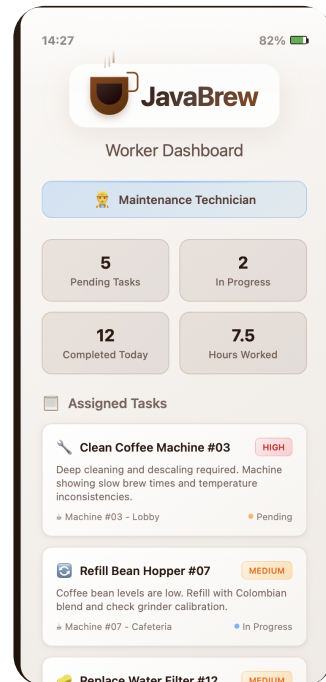


Figura 4: Dashboard dell'Amministratore.



(a) Dashboard del Cliente.



(b) Dashboard del Tecnico.

Figura 5: Dashboard per i ruoli Cliente e Tecnico.

4.3 Catalogo Prodotti e Acquisto

La schermata di selezione dei prodotti (Figura 6) funge da catalogo digitale interattivo. Il design pulito e visuale guida l'utente attraverso un processo di selezione e acquisto fluido.

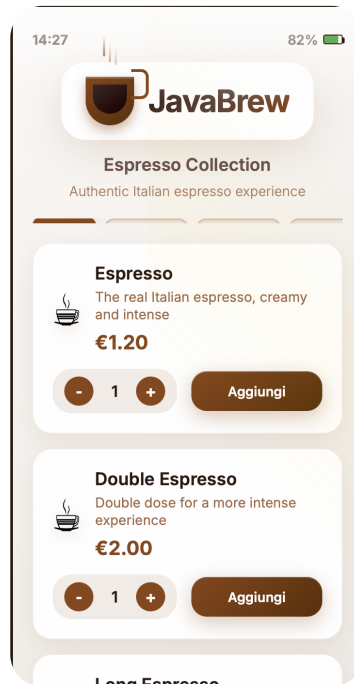


Figura 6: Schermata di selezione dei prodotti.

In sintesi, i mockup presentati dimostrano come la progettazione dell'interfaccia di **JavaBrew** sia stata guidata dai requisiti funzionali e dalle necessità specifiche di ogni attore. Queste interfacce non solo sono esteticamente gradevoli, ma sono anche funzionali e intuitive, garantendo un'esperienza utente coerente e soddisfacente. La progettazione si concentra sull'efficienza operativa e sulla facilità d'uso, elementi chiave per il successo dell'applicazione.

5 Implementazione e Testing

Questa sezione descrive le scelte implementative, le tecnologie e le strategie di testing adottate per lo sviluppo dell'applicazione **JavaBrew**. L'implementazione si basa sui modelli di progettazione definiti in fase di analisi, come illustrato dai diagrammi UML, e utilizza un insieme di tecnologie moderne per garantire robustezza, manutenibilità e testabilità del software.

5.1 Architettura e Strumenti di Sviluppo

Il progetto è stato sviluppato in **Java 17** e gestito tramite **Apache Maven** come build automation tool. L'architettura del software segue un approccio a più livelli, che garantisce una netta separazione delle responsabilità tra i vari componenti del sistema. I livelli principali, come documentato dai diagrammi UML, sono:

- **Livello Controller:** Il punto di ingresso dell'applicazione, responsabile della gestione delle richieste degli utenti e dell'invocazione della logica di business.

- **Livello Service:** Contiene la logica di business principale dell'applicazione. Orchestra le operazioni, elabora i dati e interagisce con il livello di persistenza.
- **Livello DAO (Data Access Object):** Responsabile dell'interazione con il database, astruendo completamente i dettagli della persistenza dati dal resto dell'applicazione.
- **Livello Model:** Definisce le entità del dominio che rappresentano i dati strutturati del sistema.

5.2 Livello di Persistenza: DAO e ORM Hibernate

La persistenza dei dati è gestita tramite il pattern **Data Access Object (DAO)**, che prevede un'interfaccia (es. `UserDao`) e una sua implementazione concreta (es. `UserDaoImpl`). Questo approccio disaccoppia la logica di business dalle tecnologie di persistenza specifiche.

Per l'interazione con il database, è stato utilizzato **Hibernate ORM** (versione 6.5.2), un potente framework di Object-Relational Mapping. Grazie a Hibernate e all'API standard **Jakarta Persistence (JPA)**, si è evitato di scrivere codice SQL nativo, mappando direttamente gli oggetti Java del nostro modello su tabelle di un database relazionale. Il database di produzione scelto è **PostgreSQL**.

La gestione delle connessioni al database è centralizzata nella classe `DBManager`. Questa classe agisce come un gestore statico che fornisce un punto di accesso globale e unico all'`EntityManagerFactory`, l'oggetto responsabile della creazione di `EntityManager` per interagire con il database.

```

1 package com.smartvend.app.db;
2
3 import jakarta.persistence.EntityManagerFactory;
4 import jakarta.persistence.Persistence;
5
6 public class DBManager {
7     private static EntityManagerFactory emf;
8     private static final String DEFAULT_UNIT = "smartvendPU";
9
10    public static void init() {
11        initializeDatabase(DEFAULT_UNIT);
12    }
13
14    public static void initializeDatabase(String persistenceUnitName)
15    {
16        try {
17            emf = Persistence.createEntityManagerFactory(
18                persistenceUnitName);
19        } catch (Exception e) {
20            System.err.println("Database initialization failed: " + e
21                .getMessage());
22        }
23    }
24 }

```

```

19         e.printStackTrace();
20         throw new RuntimeException("Failed to initialize database
persistence unit.", e);
21     }
22 }
23
24 public static EntityManagerFactory getEntityManagerFactory() {
25     if (emf == null) {
26         init();
27     }
28     return emf;
29 }
30
31 public static void shutdown() {
32     if (emf != null && emf.isOpen()) {
33         emf.close();
34     }
35 }
36 }

```

Listing 1: La classe DBManager che gestisce l'accesso all'EntityManagerFactory di JPA.

5.3 Livello di Logica di Business (Service)

Il livello Service contiene il nucleo della logica applicativa. Ogni servizio, come `CustomerService`, dipende dalle interfacce dei DAO per accedere ai dati, seguendo il principio di **Inversione delle Dipendenze**. Il metodo `buyItem` è un esempio emblematico di come il servizio orchestra diverse operazioni: recupera le entità, applica regole di business (controllo del saldo e della disponibilità dei prodotti), aggiorna lo stato del sistema (quantità e saldo) e infine registra la transazione.

```

1 public Transaction buyItem(long connectionId, List<Long> itemIds) {
2     // 1. Recupera la connessione e il cliente
3     Connection connection = connectionDao.getConnectionById(
connectionId);
4     if (connection == null) {
5         throw new IllegalArgumentException("Connection not found");
6     }
7     Customer customer = customerDao.getCustomerByUserId(connection.
getUserId());
8     if (customer == null) {
9         throw new IllegalArgumentException("Customer not found");
10    }
11 }

```

```

12 // 2. Calcola il prezzo totale e valida la disponibilit  degli
    articoli
13 double totalPrice = 0.0;
14 List<Item> itemsToUpdate = new ArrayList<>();
15 for (Long itemId : itemIds) {
16     Item item = itemDao.getItemById(itemId);
17     if (item == null || item.getQuantity() <= 0) {
18         throw new IllegalArgumentException("Item not found or not
    available");
19     }
20     totalPrice += item.getPrice();
21     itemsToUpdate.add(item);
22 }
23
24 // 3. Controlla il saldo del cliente PRIMA di ogni modifica
25 double initialBalance = checkBalance(customer.getId(), totalPrice
);
26
27 // 4. Se il saldo   sufficiente, aggiorna lo stato del sistema
28 for (Item item : itemsToUpdate) {
29     item.setQuantity(item.getQuantity() - 1);
30     itemDao.updateItem(item);
31 }
32 updateBalance(customer.getId(), totalPrice);
33
34 // 5. Crea e registra la transazione
35 Transaction transaction = new Transaction(
36     customer, PaymentMethod.Wallet, initialBalance,
37     initialBalance - totalPrice, createTransactionItems(
    itemsToUpdate));
38
39 return transactionService.createTransaction(transaction);
40 }

```

Listing 2: Estratto del metodo buyItem da CustomerService.

5.4 Ambiente di Esecuzione con Docker

Per garantire un ambiente di sviluppo e di esecuzione coerente e riproducibile, il database **PostgreSQL**   stato containerizzato tramite **Docker**. La containerizzazione, in questo contesto, offre vantaggi cruciali. In primo luogo, garantisce la **parit  tra gli ambienti** di sviluppo, test e produzione, eliminando la classica problematica del "funziona sulla mia macchina". Ogni sviluppatore esegue la stessa versione di PostgreSQL con la stessa configurazione. In secondo luogo, **isola le dipendenze**: il database e le sue librerie sono racchiusi nel container e non interferiscono con altri servizi o software installati sulla macchina host. Infine, l'utilizzo di un file `docker-compose.yml` per definire il servizio agisce come "Infrastructure as Code",

documentando e versionando l'ambiente necessario per eseguire l'applicazione e semplificando drasticamente il setup per i nuovi membri del team.

5.5 Strategia di Testing

La qualità del software è stata garantita attraverso una solida e completa strategia di test, che ha previsto sia **test di unità** sia **test di integrazione**. Questa strategia a più livelli è fondamentale perché permette di verificare il sistema da diverse prospettive: la correttezza logica dei singoli componenti e la loro corretta collaborazione all'interno dell'architettura complessiva. Gli strumenti utilizzati, come definiti nel `pom.xml`, hanno permesso di automatizzare e validare il corretto funzionamento del codice a diversi livelli.

5.5.1 Test di Unità e Mocking

I test di unità sono stati scritti utilizzando **JUnit 5** per verificare la correttezza dei singoli componenti in isolamento. Per testare il livello Service senza dipendere dal livello di persistenza, è stato utilizzato intensivamente il framework di mocking **Mockito**. Questo ha permesso di simulare il comportamento dei DAO (tramite l'annotazione `@Mock`) e di iniettarli nel servizio da testare (tramite `@InjectMocks`), concentrando i test esclusivamente sulla logica di business. Questo isolamento è vitale: permette di eseguire centinaia di test in pochi secondi, senza la latenza di un database, e di verificare ogni singolo percorso logico e caso limite del servizio in modo deterministico.

```
1 public class CustomerServiceTest {
2     @Mock
3     private CustomerDaoImpl customerDao;
4     @Mock
5     private ConnectionDaoImpl connectionDao;
6     @Mock
7     private ItemDaoImpl itemDao;
8     @Mock
9     private TransactionService transactionService;
10    // ... altri mock ...
11
12    @InjectMocks
13    private CustomerService customerService;
14
15    // Oggetti mock preparati nel metodo @BeforeEach
16    private Customer mockCustomer;
17    private Item mockItem1;
18
19    @BeforeEach
20    public void setUp() {
21        MockitoAnnotations.openMocks(this);
22        // ... inizializzazione di mockCustomer, mockItem1, etc.
```

```

23     mockCustomer = new Customer(2L, new User(1L, "...", "Jhon", "
Doe", "..."), 100.0);
24     mockItem1 = new Item(30L, "Soda", "...", 1, 10, 1.50, 3,
ItemType.Bottle);
25     }
26
27     @Test
28     @DisplayName("Test buyItem method success with single item")
29     void testBuyItemSuccessSingleItem() {
30         // Arrange: Prepara i dati e il comportamento dei mock
31         Connection mockConnection = new Connection(mockCustomer.
getUser(), ...);
32         Transaction expectedTransaction = new Transaction(
mockCustomer, ...);
33
34         when(connectionDao.getConnectionById(anyLong())) .thenReturn (
mockConnection);
35         when(customerDao.getCustomerByUserId(anyLong())) .thenReturn (
mockCustomer);
36         when(itemDao.getItemById(30L)) .thenReturn(mockItem1);
37         when(transactionService.createTransaction(any(Transaction.
class)))
38             .thenReturn(expectedTransaction);
39
40         // Act: Esegui il metodo da testare
41         Transaction result = customerService.buyItem(1L, List.of(30L)
);
42
43         // Assert: Verifica il risultato e le interazioni con i mock
44         assertNotNull(result);
45         assertEquals(expectedTransaction, result);
46         assertEquals(9, mockItem1.getQuantity()); // Verifica
decremento quantita'
47
48         verify(itemDao, times(1)).updateItem(mockItem1);
49         verify(customerDao, times(1)).updateCustomer(mockCustomer);
50         verify(transactionService, times(1)).createTransaction(any(
Transaction.class));
51     }
52 }

```

Listing 3: Test unitario per il metodo `buyItem` con JUnit 5 e Mockito.

5.5.2 Test di Integrazione

I test di integrazione sono stati realizzati per verificare la corretta collaborazione tra i diversi livelli dell'applicazione, in particolare tra il Service Layer, il DAO Layer e il database. Per

questi test, è stato utilizzato un database in-memory **H2**, che ha permesso di eseguire l'intero stack di persistenza in un ambiente veloce e controllato. Questi test sono cruciali per individuare errori che i test di unità non possono cogliere, come problemi nelle annotazioni di mapping di JPA, errori nella gestione delle transazioni, o query che potrebbero non essere compatibili con il dialetto SQL specifico di PostgreSQL. Validare queste interazioni in un ambiente controllato, prima del deployment, è un passo fondamentale per garantire la robustezza del sistema.

5.5.3 Analisi della Copertura del Codice

Per monitorare l'efficacia e la completezza della suite di test, è stato integrato il plugin **JaCoCo** in Maven. L'esecuzione combinata dei test di unità e di integrazione ha permesso di raggiungere e mantenere una **copertura del codice superiore all'80%**. Questo risultato quantitativo fornisce una metrica oggettiva della qualità del software, dimostrando che la maggior parte della logica implementata è stata verificata da test automatici. Inoltre, l'analisi della copertura aiuta a identificare il "codice morto" (parti di codice mai eseguite) e a garantire che i percorsi logici critici, specialmente quelli relativi alla gestione degli errori, siano stati adeguatamente testati, guidando così il miglioramento continuo della qualità.